

VirtuEx 시스템 감사 보고서 (26차)

VirtuEx System Audit Report (26th)

- 프로젝트: VirtuEx (실시간 모의 주식/암호화폐 거래 플랫폼)
- 감사 차수: 26차 감사
- 감사일: 2026-03-31
- 감사 범위: API 보안, 인증/인가, 에러 처리, CSP/HTTP 헤더, WebSocket 보안, 프론트엔드 보안, 파일 업로드, 의존성, 시크릿/설정, 데이터베이스 보안 (전체 재감사)
- 감사 방법: 전체 소스 코드 정적 분석 (백엔드 7개 마이크로서비스 + 프론트엔드 Next.js)
- 심각도 기준: Critical(즉시 수정) / High(1주 내) / Medium(2주 내) / Low(개선 권장)

이전 감사 대비 수정 현황 (25차 → 26차)

이전 이슈	상태	비고
[SEC-H-03] 응답 바디에 accessToken 노출	미수정 (의도적)	WebSocket handshake 필요. httpOnly 쿠키가 주 인증 경로이며 바디 토큰은 WebSocket 전용 보조 경로로 유지
[SVC-M-01] 서비스 간 HTTP 통신 HTTPS 미적용	미수정 (의도적)	Docker 내부 네트워크 한정. 단일 호스트 Docker Compose 환경에서 리스크 대비 비용 비합리적
[CSP-25-01] CSP middleware 개발모드 호환성	수정 유지	middleware.ts에서 개발 모드 분기 정상 적용 확인
[HYD-25-01] Layout hydration mismatch	수정 유지	raw <script> + suppressHydrationWarning 정상 적용 확인
[WS-25-01] WebSocket Strict Mode 이종 연결	수정 유지	autoConnect: false + disposed 플래그 정상 적용 확인

25차 미수정 2건 의도적 유지, 수정 3건 정상 유지 확인

26차 감사 결과 요약

심각도	발견 건수
Critical	0
High	2
Medium	5
Low	6
합계	13

1. API 보안 감사

[SEC-26-01] 다수 프록시 엔드포인트에서 @Body() body: unknown 사용 — DTO 검증 부재

- 심각도: High

- 파일 및 위치:
 - backend/services/api-gateway/src/proxy/community-proxy.controller.ts (line 106, 134, 236, 313, 341)
 - backend/services/api-gateway/src/proxy/strategy-proxy.controller.ts (line 105, 133, 235)
 - backend/services/api-gateway/src/proxy/follow-proxy.controller.ts (line 44, 227)
 - backend/services/api-gateway/src/proxy/portfolio-proxy.controller.ts (line 47, 80)
 - backend/services/api-gateway/src/proxy/price-alert-proxy.controller.ts (line 37)
 - backend/services/api-gateway/src/proxy/copy-trade-proxy.controller.ts (line 40, 59)
 - backend/services/api-gateway/src/proxy/profile-proxy.controller.ts (line 51, 68)
 - backend/services/api-gateway/src/proxy/announcement-proxy.controller.ts (line 141, 178, 242, 341, 404)
 - backend/services/api-gateway/src/proxy/ai-proxy.controller.ts (line 47, 65)
 - backend/services/api-gateway/src/proxy/statistics-proxy.controller.ts (line 41)
- 설명: @Body() body: unknown 으로 선언된 엔드포인트는 NestJS의 ValidationPipe (whitelist: true, forbidNonWhitelisted: true)가 적용되지 않습니다. unknown 타입에는 class-validator 데코레이터가 없으므로 모든 필드가 그대로 하위 서비스로 전달됩니다. 하위 서비스에 자체 검증이 있더라도, 게이트웨이에서 조기 차단하지 않으면 불필요한 네트워크 트래픽과 하위 서비스 부하가 발생합니다. 특히 커뮤니티 게시물 (createPost), 프로필 수정 (updateProfile), 비밀번호 변경 (changePassword), 가격 알림 생성, 카피 트레이딩 시작 등 데이터 변경 엔드포인트가 해당됩니다.
- 권장 수정: 각 엔드포인트에 대응하는 DTO 클래스를 생성하고 class-validator 데코레이터를 적용합니다. auth, order 관련 엔드포인트에서 이미 DTO 패턴(LoginDto , PlaceOrderDto 등)이 적용되어 있으므로 동일 패턴으로 확장합니다.

[SEC-26-02] check-duplicate 엔드포인트 field 파라미터 검증 부재

- 심각도: Medium
- 파일: backend/services/api-gateway/src/proxy/auth-proxy.controller.ts (line 333-344)
- 설명: @Query('field') field: string 이 서버 측 허용 값(email , username) 검증 없이 그대로 user-auth 서비스로 전달됩니다. 하위 서비스에서 검증하더라도, 게이트웨이 레벨에서 화이트리스트 검증이 없으면 의도치 않은 필드 참조가 가능합니다.
- 권장 수정: 게이트웨이에서 field 가 'email' 또는 'username' 인지 검증하고, 그 외 값은 BadRequestException 을 반환합니다. 또는 IsIn(['email', 'username']) 데코레이터가 적용된 DTO 를 사용합니다.

[SEC-26-03] 가입 성공 시 WebSocket 전체 브로드캐스트에 사용자 정보 포함

- 심각도: Low
- 파일: backend/services/api-gateway/src/proxy/auth-proxy.controller.ts (line 88-96)
- 설명: 회원가입 성공 시 server.emit('notification:registration-request', ...) 이벤트로 username , name 을 연결된 모든 WebSocket 클라이언트에 브로드캐스트합니다. 관리자뿐 아니라 일반 사용자의 소켓에도 전송되어, 가입 신청자의 이름과 아이디가 불필요하게 노출됩니다.
- 권장 수정: server.emit() 대신 관리자 사용자에게만 전송(ADMIN / SYSTEM role 사용자의 소켓에만 emit)하도록 변경합니다. ChatGateway 에 notifyAdmins(event, data) 같은 메서드를 추가하는 것을 권장합니다.

2. 인증/인가 감사

[SEC-26-04] CSRF 보호 미구현 (httpOnly 쿠키 인증 환경)

- 심각도: Medium
- 파일: 프로젝트 전체 (CSRF 토큰 관련 코드 부재)
- 설명: auth.ts 의 마이그레이션 계획(Step 3)에서 CSRF 보호 필요성을 인지하고 있으나, 현재 구현되지 않았습니다. httpOnly 쿠키가 주 인증 방식이므로 SameSite=Lax 설정이 기본 CSRF 방어를 제공하지만, Lax 는 GET 요청의 cross-site navigation에서 쿠키를 전송하므로 완전하지 않습니다. 특히 주문 생성, 자금 입출금, 계정 설정 변경 등 상태 변경 API에 대한 CSRF 공격이 가능할 수 있습니다.
- 권장 수정: Double-submit cookie 패턴 또는 Synchronizer token 패턴으로 CSRF 보호를 구현합니다. SameSite 를 Strict 로 강화하거나, 상태 변경 요청에 커스텀 헤더(X-CSRF-Token)를 요구하는 방법을 권장합니다.

[SEC-26-05] 관리자 경로 미들웨어에서 JWT 쿠키 존재만 확인 (서명 미검증)

- 심각도: Low
- 파일: frontend/src/middleware.ts (line 58-63)
- 설명: 관리자 경로(/admin/*) 접근 시 access_token 쿠키의 존재 여부만 확인하고, JWT 서명이나 역할을 검증하지 않습니다. Edge Runtime에서 JWT 검증에 제약이 있을 수 있으나, 만료된 토큰이나 변조된 값으로도 관리자 페이지에 진입할 수 있습니다. 실제 API 호출은 백엔드 AdminRolesGuard 에 의해 보호되지만, 관리자 UI 자체가 비인가 사용자에게 노출됩니다.
- 권장 수정: Edge Runtime 호환 JWT 디코딩(서명 검증은 아니더라도 role 필드 확인)을 추가하거나, 최소한 jose 라이브러리를 사용하여 Edge Runtime에서 JWT 서명 검증을 수행합니다.

[SEC-26-06] 채팅방 강퇴/삭제에서 req.user.role 기반 인라인 권한 검사

- 심각도: Low
- 파일: backend/services/api-gateway/src/proxy/chat-proxy.controller.ts (line 251-253, 342-344)
- 설명: kickUser 와 deleteRoom 에서 if (user.role !== 'ADMIN' && user.role !== 'SYSTEM') 인라인 검사를 수행합니다. 이는 AdminRolesGuard 와 동일한 로직을 컨트롤러에 중복 구현한 것으로, 일관성이 떨어지고 향후 역할 체계 변경 시 누락될 위험이 있습니다.
- 권장 수정: @UseGuards(AdminRolesGuard) 를 해당 메서드에 적용하여 가드 기반 접근 제어로 통일합니다.

3. 에러 처리 감사

[SEC-26-07] Record<string, any> 타입 사용 잔존 (copy-trade, ai, market 컨트롤러)

- 심각도: Low
- 파일:
 - backend/services/api-gateway/src/proxy/copy-trade-proxy.controller.ts (line 41, 63, 84, 99, 119, 141)
 - backend/services/api-gateway/src/proxy/ai-proxy.controller.ts (line 48)
 - backend/services/api-gateway/src/proxy/market-proxy.controller.ts (line 31, 62)
 - backend/services/api-gateway/src/proxy/news-proxy.controller.ts (line 40)
- 설명: 25차 감사에서 portfolio-proxy.controller.ts 의 Record<string, any> → AuthenticatedRequest 교체가 완료되었으나, copy-trade-proxy.controller.ts (6개소)와 ai-proxy.controller.ts (1개소)에 (req as Record<string, any>).user?.id 패턴이 잔존합니다.

market-proxy.controller.ts 와 news-proxy.controller.ts 에서는 `HttpException` 생성 시 `Record<string, any>` 를 사용합니다. 코딩 규칙에서 `Any` 타입을 지양하도록 명시되어 있습니다.

- **권장 수정:** `copy-trade-proxy.controller.ts` 와 `ai-proxy.controller.ts` 에서 `AuthenticatedRequest` 인터페이스를 import하여 사용합니다. `HttpException` 인자에는 `Record<string, unknown>` 을 사용합니다.

4. CSP/HTTP 헤더 감사

[SEC-26-08] CSP `style-src` 에 '`unsafe-inline`' 사용

- **심각도:** Medium
- **파일:**
 - `frontend/src/middleware.ts` (line 43)
 - `backend/services/api-gateway/src/main.ts` (line 43)
- **설명:** 프론트엔드 미들웨어와 백엔드 Helmet 모두 `style-src 'self' 'unsafe-inline'` 을 설정하고 있습니다. '`unsafe-inline`' 은 XSS 공격 시 인라인 스타일 삽입을 허용합니다. TipTap 에디터 등 외부 라이브러리가 인라인 스타일을 사용하기 때문에 현재 필요하지만, nonce 기반 스타일 정책으로 전환하면 보안을 강화할 수 있습니다.
- **권장 수정:** 스타일에도 nonce 기반 CSP를 적용하거나, 최소한 현재 상태가 의도적임을 주석으로 명시합니다. TipTap 등 외부 라이브러리의 인라인 스타일 의존성 제거가 선행되어야 합니다.

5. WebSocket 보안 감사

[SEC-26-09] Price Gateway에서 인증 실패 시 익명 연결 허용 + 브로드캐스트 수신 가능

- **심각도:** Medium
- **파일:** `backend/services/api-gateway/src/gateway/price.gateway.ts` (line 94-104)
- **설명:** `PriceGateway` 에서 유효하지 않은 JWT 토큰을 제공한 클라이언트를 익명 사용자로 강등하여 연결을 유지합니다(line 94 catch 블록). 이는 의도된 동작(비인증 사용자도 제한적 가격 조회 가능)이지만, `broadcastPriceBatch()` (line 212)는 모든 연결된 클라이언트(`this.server.emit`)에 가격 업데이트를 전송하므로, 익명 사용자가 구독하지 않은 심볼의 가격 데이터도 수신할 수 있습니다.
- **권장 수정:** `broadcastPriceBatch()` 에서 `this.server.emit` 대신 구독된 room에만 전송하도록 변경하거나, 익명 사용자에게 대한 배치 수신을 별도 제어합니다.

6. 프론트엔드 보안 감사

[SEC-26-10] `dangerouslySetInnerHTML` 사용 — `DOMPurify` 적용 확인됨 (양호)

- **심각도:** (양호 — 이슈 아님)
- **파일:** `frontend/src/app/(main)/community/[id]/page.tsx` (line 402)
- **설명:** 커뮤니티 게시물 HTML 렌더링에 `dangerouslySetInnerHTML` 을 사용하지만, `DOMPurify.sanitize()` 가 적용되어 있으며, 허용 태그/속성이 엄격하게 제한되어 있습니다(`ALLOWED_TAGS` , `FORBID_ATTR: ['onerror', 'onload', 'onclick', 'onmouseover', 'src']` , `ALLOW_DATA_ATTR: false`). `img` 태그와 `src` 속성이 모두 차단되어 XSS 벡터가 효과적으로 제거되었습니다.
- **비고:** `layout.tsx` 의 `dangerouslySetInnerHTML` 은 빌드 타임 고정 문자열이며 nonce가 적용되어 있어 안전합니다.

7. 파일 업로드 감사

[SEC-26-11] 커뮤니티/공지사항 첨부파일 업로드 — 게이트웨이 레벨 파일 타입 검증 부재

- 심각도: Medium
 - 파일:
 - `backend/services/api-gateway/src/proxy/community-proxy.controller.ts` (line 335-357, `uploadAttachment`)
 - `backend/services/api-gateway/src/proxy/community-proxy.controller.ts` (line 307-329, `uploadImage`)
 - 설명: 커뮤니티 첨부파일 업로드(`uploadAttachment`)와 이미지 업로드(`uploadImage`)에서 `@Body()` `body: unknown` 으로 body를 수신하여 검증 없이 하위 서비스로 전달합니다. 프론트엔드(`RichEditor.tsx`)에서 MIME 타입과 크기 검증을 수행하지만, 게이트웨이에서는 공지사항 첨부과 달리 사전 검증이 없습니다. 디렉토리 트래버설은 `path.basename()` 처리로 방어되어 있으나, 파일 타입/크기 검증은 하위 서비스에 의존합니다.
 - 권장 수정: 공지사항의 `uploadAttachment` (line 345-358)에 적용된 크기 사전 검증 패턴을 커뮤니티 업로드에도 동일하게 적용합니다. MIME 타입 화이트리스트 검증도 게이트웨이에 추가합니다.
-

8. 의존성 보안 감사

[SEC-26-12] 시드 스크립트에 하드코딩된 관리자 비밀번호

- 심각도: High
 - 파일:
 - `scripts/seed.ts` (line 99, 110)
 - `scripts/seed-test-users.ts` (line 31)
 - 설명: 시드 스크립트에 관리자 계정의 비밀번호가 평문으로 하드코딩되어 있습니다(`admin1234!` , `ehgml5516!`). 이 스크립트가 Git에 포함되어 있으므로, 리포지토리 접근 권한이 있는 누구나 관리자 비밀번호를 알 수 있습니다. 프로덕션 환경에서 이 비밀번호가 변경되지 않으면 즉각적인 보안 위험이 됩니다.
 - 권장 수정: 시드 비밀번호를 환경 변수(`SEED_ADMIN_PASSWORD`)로 분리하고, 스크립트에서 환경 변수가 없으면 임의의 강력한 비밀번호를 생성하도록 변경합니다. 프로덕션 배포 전 모든 시드 계정의 비밀번호를 반드시 변경하도록 배포 체크리스트에 포함합니다.
-

9. 시크릿/설정 감사

[SEC-26-13] `.env.example` 의 Redis 비밀번호가 약한 기본값

- 심각도: Low
 - 파일: `.env.example` (line 28-29)
 - 설명: `.env.example` 에서 Redis 비밀번호가 `redis` 로 설정되어 있습니다. 개발 환경용이지만, `.env.example` 을 그대로 `.env` 로 복사하여 사용하는 경우(특히 스테이징 환경) 약한 비밀번호가 그대로 적용될 수 있습니다. PostgreSQL과 `JWT_SECRET` 은 `<CHANGE_ME_IN_PRODUCTION>` 플레이스홀더를 사용하지만 Redis는 실제 값이 입력되어 있습니다.
 - 권장 수정: Redis 비밀번호도 `<CHANGE_ME_IN_PRODUCTION>` 플레이스홀더로 변경합니다.
-

10. 데이터베이스 보안 감사

[SEC-26-14] `$queryRawUnsafe` 사용 (`copy-trade.service.ts`)

- 심각도: Low (현재 안전하지만 주의 필요)
- 파일: `backend/services/portfolio/src/domain/services/copy-trade.service.ts` (line 346)

- **설명:** `$queryRawUnsafe` 를 사용하여 `FOR UPDATE` 잠금 쿼리를 실행합니다. 현재 `$1` 파라미터 바인딩을 사용하고 있어 SQL 인젝션 위험은 없습니다. 그러나 `$queryRawUnsafe` 는 이름 그대로 안전하지 않은 API이며, 향후 코드 수정 시 문자열 보간으로 변경될 위험이 있습니다.
- **권장 수정:** 가능하다면 `$queryRaw` (tagged template literal)로 교체합니다: `tx.$queryRaw`SELECT ... WHERE "id" = ${config.id} FOR UPDATE``. 이 형식은 Prisma가 자동으로 파라미터 바인딩을 처리합니다.

종합 평가

보안 수준: 양호 (Good)

VirtuEx 프로젝트는 25차 감사까지의 지속적인 보안 강화를 통해 전체적으로 양호한 보안 수준을 유지하고 있습니다.

강점:

- httpOnly 쿠키 기반 인증 + `sanitizeAndForwardCookies` 메서드로 Secure/HttpOnly/SameSite 플래그 보장
- 전역 `ValidationPipe` (`whitelist: true`, `forbidNonWhitelisted: true`) + 주요 엔드포인트 (auth, order)에 DTO 적용
- 포괄적인 CSP 정책 (nonce 기반 `script-src` + `strict-dynamic`)
- WebSocket 인증 (JWT 검증), 이벤트 화이트리스트, 메시지 길이 제한, 타이핑 쓰로틀
- `AllExceptionsFilter` 에서 스택 트레이스 제거 및 일관된 에러 응답 형식
- DOMPurify 기반 XSS 방어 (엄격한 태그/속성 화이트리스트)
- 디렉토리 트래버설 방지 (`path.basename()`)
- 내부 서비스 인증 (`InternalAuthGuard` + `timingSafeEqual`)
- Circuit Breaker 패턴으로 서비스 장애 전파 방지
- CORS에서 내부 헤더(`x-internal-token` , `x-user-id`) 외부 설정 차단
- 공개 포트폴리오 데이터 마스킹 (절대 수량 → 비율/범주)
- Swagger 문서 프로덕션 비활성화

개선 필요:

- 프록시 컨트롤러 대다수에서 `@Body()` `body: unknown` → DTO 검증 필요 (High)
- 시드 스크립트 하드코딩 비밀번호 분리 필요 (High)
- CSRF 보호 구현 필요 (Medium)
- 커뮤니티 파일 업로드 게이트웨이 레벨 검증 추가 (Medium)
- `Record<string, any>` 타입 잔존 정리 (Low)

본 보고서는 2026-03-31 기준 소스 코드 정적 분석 결과입니다.